Programación II - Trabajo Práctico Integrador
1er. Cuatrimestre 2026
SEGUNDA PARTE

Fecha de presentación: 09/10/2026(subido al Campus)

Fecha límite de entrega: 23/10/2026 (por el Campus)

En esta segunda parte deben entregar la implementación, el diagrama de clases actualizado, el análisis de complejidad en donde se pida, y el IREP para cada tipo de datos modelado en su solución. Para poder empezar con la segunda parte deben tener aprobado el diseño presentado en la primera parte.

Requerimientos técnicos:

i- Grupos: **El mismo grupo de la primera parte del TP.** Si hay alguna modificación debe ser aprobada por sus docentes de la comisión.

ii- Se deben utilizar donde sea conveniente las herramientas de Tecnologías Java que se vieron en la materia. Al menos una vez deben usarse:

- *Stringbuilder,* cuyo uso debe basarse en la necesidad de modificar el string.
- *Iteradores y Foreach* para recorrer las colecciones de Java

iii- Se deberá utilizar en el desarrollo del trabajo **herencia y polimorfismo,** y al menos 2 de estos conceptos: **sobreescritura, sobrecarga e interfaces**. Como también, en los casos que corresponda, se deberá implementar **clases/métodos abstractos**.

iv- **En el informe (documento) se debe explicar donde utilizaron estos conceptos.**

v- **Escribir el *IREP* de la representación elegida para la implementación de cada TAD**. Debe ser parte de la documentación.

Por otro lado, desde la materia se proveerá

a) Una Interfaz para que se utilice como base para la implementación de la clase principal Billetera, con la explicación de cada método. **NO SE DEBE MODIFICAR**.
b) Un código cliente con datos para crear los objetos, **NO SE DEBE MODIFICAR**.
c) Una clase de testeo (junit). Será condición necesaria para aprobar esta parte del trabajo que tanto el código cliente como el test se ejecuten sin errores. **NO SE DEBE MODIFICAR**.

- Además de pasar el test de *junit* suministrado junto con el TP, en la corrección se testean los ejercicios con otro junit adicional, por lo que se recomienda que el grupo arme un conjunto propio de testeo acorde a su implementación, antes de entregar el TP. Puede entregarlo también si lo desea.

La entrega se realiza subiendo al Campus el proyecto de Java con su implementación **(Seleccionar solamente los archivos .java)** También, se debe subir la documentación con los puntos pedidos previamente por escrito en un archivo PDF, junto con el proyecto.

Consideraciones importantes para la implementación y la documentación del trabajo:

La implementación debe responder a su diseño presentado en la Primera Parte ***teniendo en cuenta las correcciones que se indicaron/indicarán a cada grupo*. Además, será condición necesaria para aprobar que se cumpla con:**

- Deberá correr satisfactoriamente con el código cliente entregado
- Deberá pasar satisfactoriamente el test junit proporcionado.
- Deberá aprovechar correctamente las estructuras de datos elegidas.
- El código deberá tener implementado el método ***toString*** de la clase principal, lo que implica que se deban implementar los toString de los TADs relacionados.
- Se deberá usar herencia, polimorfismo y abstracción.

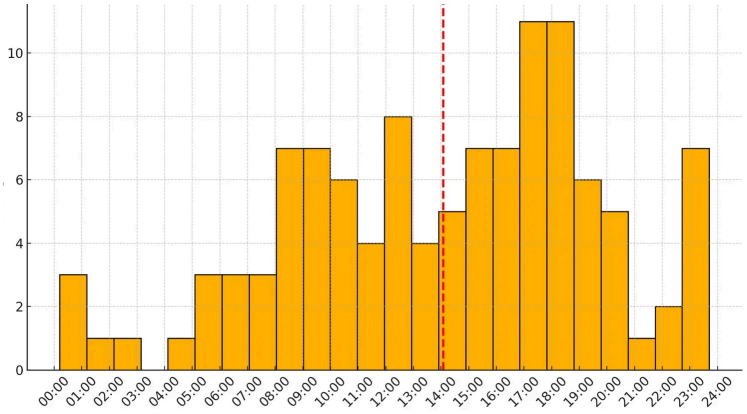
Algunas definiciones, modificaciones y aclaraciones al enunciado de la primera parte.

Definiciones:

- Al pasar una planta a inactiva y luego activa, se debe dejar un registro del periodo de tiempo que estuvo inactiva. Cada planta debe conocer todos los momentos en los que estuvo inactiva.

Modificaciones y aclaraciones al enunciado de la primera parte:

- 1. El punto 1 se divide en 2 partes:
  - 1.a) registro de una planta nueva
  - 1.b) consulta de las plantas existentes. Que devuelve un lista con el detalle de cada planta:
    - cada ítem debe respetar el formato: "`{NOMBRE} ({ACTIVA|INACTIVA}): {FECHA_PUESTA_EN_FUNCIONAMIENTO}`"
    - Por ejemplo: "`Atucha II (ACTIVA): 2014-06-27T09:00:00`"
- 7. El histograma cuantifica la cantidad de días (entre las 2 fechas indicadas por parámetro) en que la planta cumplió la cuota para cada hora. Esto nos permite ver la capacidad de generación por hora entre 2 fechas. Permitiendo generar gráficas como la siguiente:



- 10. Devolver una lista con todas las plantas eólicas indicando las horas en las que cada planta nunca obtuvo la velocidad mínima de viento necesaria para poder generar energía. Debe devolver un String con el siguiente formato de ejemplo:
  “””
  ```
  Plantas Eolicas: Horas con poco viento
   - Planta 1:  5 m/s
          07: 2 m/s
          11: 3 m/s
          12: 1 m/s
          13: 0 m/s
   - Mi Super Planta:  4 m/s
          07: 2 m/s
          11: 3 m/s
  ```
  “””

Se agregan las siguientes funcionalidades:

- 11. Limpiar registros de monitoreo sin datos y devolver la cantidad de registros borrados. Se deben borrar todos los registros de monitoreo que tengan cero energía generada y cero incidencias.
- 12. Devolver una lista con las plantas y cuánta energía generó cada una hasta el momento ordenado de mayor a menor energía generada. Para cada planta se debe respetar el siguiente formato: “{NOMBRE_PLANTA}: {TOTAL_GENERADO}”
- 13. Dado un fecha y hora, se debe devolver los nombres de todas las plantas que estuvieron inactivas en ese momento.

Con las observaciones anteriores, se deberá implementar el TP, teniendo en cuenta las siguientes condiciones:

- Se debe poder imprimir “SmartGrids” mostrando sus datos en formato adecuado para poder comprenderlo. Se espera visualizar el estado interno del sistema de una forma legible.

También se deberá entregar en el documento el siguiente análisis de la complejidad:
Explicar la complejidad lograda y justificar por medio de Álgebra de Órdenes para el punto:

- *7. Calcular histograma de cumplimiento de cuota por hora.*

(Pegar el código de todas las funciones necesarias y analizarlas en el mismo informe. Esta es la única porción de código que debe haber en el informe)

Test (JUnit):

Se habilitará el archivo de test en el Moodle, junto a este enunciado, de donde deberán descargarlo. **Se avisará en cuanto esté disponible**.

Código Cliente:

Ya está habilitado junto con este enunciado, en el espacio del Moodle para el TP, de donde deberán descargarlo.

Interfaz:

Ya está habilitado junto con este enunciado. Se utilizará como Interfaz para crear la clase SmartGrids.